



DOCKER SCHEMATIC.

A blueprint-style reference for images, layers, containers, and the daemon that runs them — how a Dockerfile becomes a running process, drawn out.

SHEETS 01–13 SCOPE ENGINE + IMAGES + NETWORKING + COMPOSE REV 2026.1

DKR–01

What Docker Is

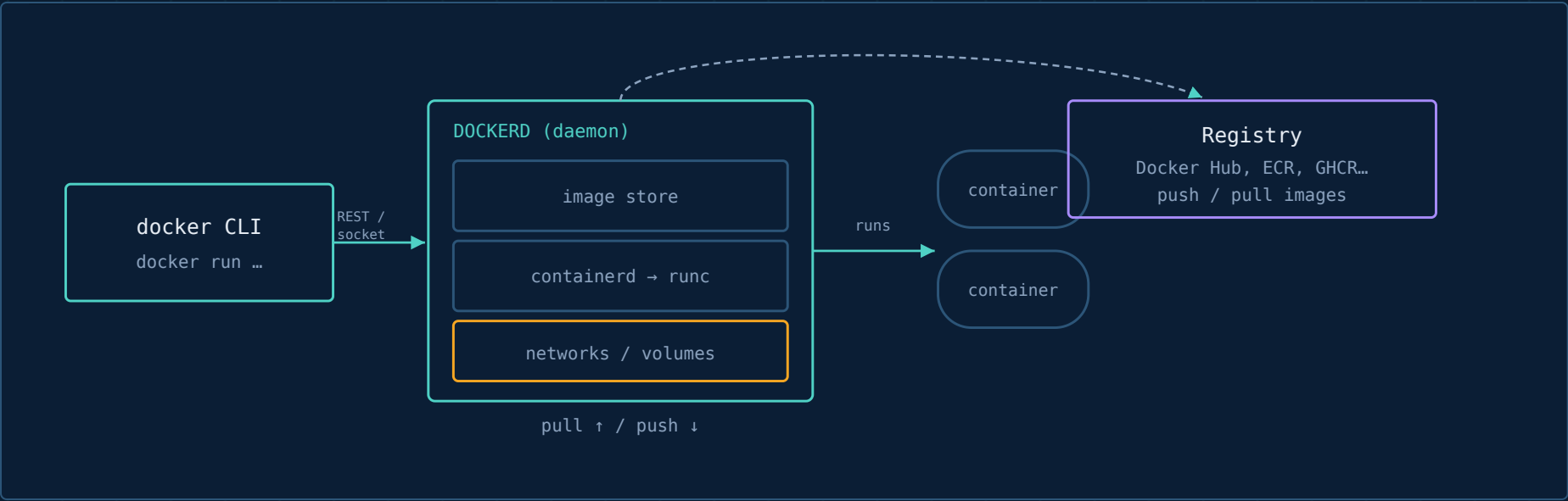
Docker packages an application with everything it needs to run — code, runtime, libraries, system tools — into an **image**: a single, portable artifact. A **container** is that image, running as an isolated process on the host, using Linux kernel features (namespaces, cgroups) instead of a full virtual machine.

- ▶ **Not a VM.** Containers share the host kernel — no guest OS to boot — so they start in milliseconds and carry far less overhead than a VM.
- ▶ **Build once, run anywhere.** An image built on a laptop runs identically on a CI runner or a production server, because the image bundles its own filesystem.
- ▶ **Immutable images, disposable containers.** You don't patch a running container — you rebuild the image and replace the container.
- ▶ **Isolation, not by default security.** Namespaces isolate what a process can see; cgroups limit what it can use. Real hardening still needs deliberate configuration.

DKR–02

Engine Architecture

The `docker` command you type is a thin client. It talks over a REST API (usually a Unix socket) to the **daemon**, which does all the real work — pulling images, building layers, starting containers.

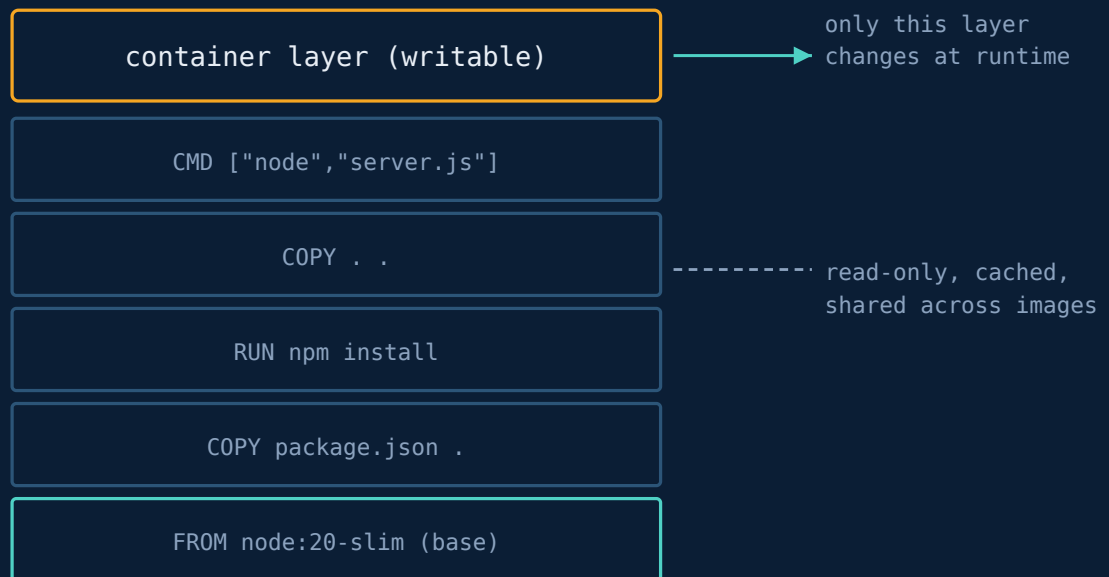


PIECE	JOB
docker CLI	Sends commands to the daemon's API — it holds no state itself.
dockerd	The daemon: builds images, manages storage, networks, and volumes.
containerd	Manages container lifecycle (start/stop/pause) below dockerd.
runc	The low-level OCI runtime that actually creates the namespaced process.
Registry	Stores and distributes images by name:tag (e.g. <code>nginx:1.27</code>).

DKR-03

Images: Layers & Union Filesystem

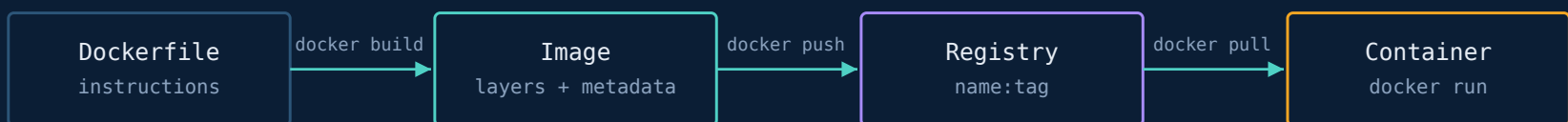
An image is a stack of read-only layers, each one a diff from the layer below — one per Dockerfile instruction that touches the filesystem. A container adds one thin writable layer on top.



- ▶ **Layer caching.** Docker reuses a cached layer if its instruction and context haven't changed — order Dockerfile instructions from least- to most-frequently-changing.
- ▶ **Copy dependency manifests first.** `COPY package.json .` then `RUN npm install` before `COPY . .` — code changes won't bust the dependency-install cache.
- ▶ **Union filesystem** (overlay2 on Linux) presents all layers as one merged view; writes go to the top writable layer only (copy-on-write).

DKR-04

Dockerfile → Image → Container

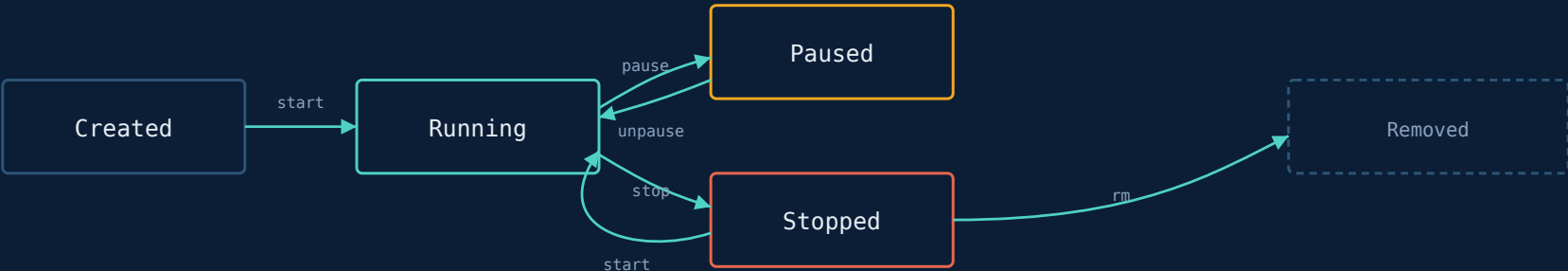


```
DOCKERFILE
FROM node:20-slim
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
```

```
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

DKR-05

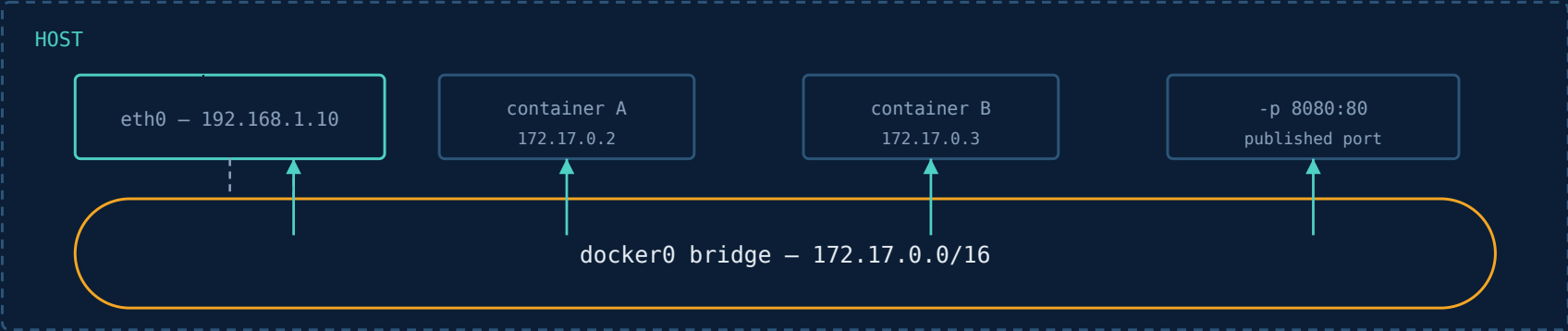
Container Lifecycle



COMMAND	EFFECT
<code>docker create</code>	Allocates the container filesystem/config but doesn't start the process.
<code>docker start / run</code>	Starts (create + start) the container's main process.
<code>docker pause / unpause</code>	Freezes/resumes all processes in the container via cgroups freezer — no signal sent.
<code>docker stop</code>	Sends SIGTERM, waits (default 10s), then SIGKILL if still running.
<code>docker rm</code>	Deletes the stopped container and its writable layer — data not in a volume is gone.

DKR-06

Networking Modes



DRIVER	BEHAVIOR
bridge (default)	Private virtual network on the host; containers get their own IP, reach each other by name if on a user-defined bridge.
host	Container shares the host's network namespace directly — no port mapping, no isolation.
none	No networking at all — fully isolated.
overlay	Spans multiple Docker hosts (Swarm) — containers on different machines reach each other as if on one LAN.

NOTE

The default bridge network doesn't do DNS-based name resolution between containers — create a user-defined bridge (`docker network create`) so containers can reach each other by container name.

DKR-07

Volumes, Bind Mounts, tmpfs

A container's writable layer disappears with the container. Anything that must outlive it — or be shared with the host — needs a mount.

TYPE	LIVES WHERE	MANAGED BY	TYPICAL USE
Volume	Docker-managed area on the host (<code>/var/lib/docker/volumes</code>)	Docker	Persistent app data — databases, uploads. The recommended default.
Bind mount	Any host path you choose	You	Local development — mount source code straight into the container.
tmpfs mount	Host memory only, never written to disk	Docker	Sensitive or highly transient data (secrets, caches).

```
VOLUMES

# named volume, Docker-managed
docker run -v pgdata:/var/lib/postgresql/data postgres

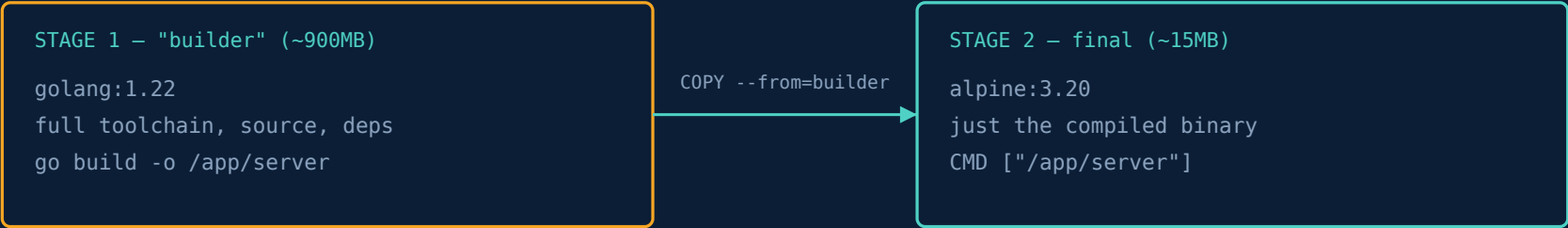
# bind mount, host path -> container path
docker run -v $(pwd)/src:/app/src node:20 npm run dev

# tmpfs, memory only
docker run --tmpfs /app/cache myapp
```

DKR-08

Multi-Stage Builds

Build tools (compilers, dev dependencies) bloat an image and widen its attack surface. Multi-stage builds compile in one stage, then copy only the finished artifact into a lean final image.



```
DOCKERFILE

FROM golang:1.22 AS builder
WORKDIR /src
COPY . .
RUN go build -o /app/server

FROM alpine:3.20
COPY --from=builder /app/server /app/server
CMD ["/app/server"]
```

DKR-09

Docker Compose

Compose declares a multi-container application — services, the network that joins them, and their volumes — in one YAML file, brought up with one command.



```
COMPOSE.YAML
```

```
services:
  web:
    build: .
    ports:
      - "8080:3000"
    environment:
      DB_HOST: db
    depends_on:
      - db

  db:
    image: postgres:16
    volumes:
      - pgdata:/var/lib/postgresql/data
    environment:
      POSTGRES_PASSWORD: example

volumes:
  pgdata:
```

`docker compose up -d`

`docker compose down`

`docker compose logs -f web`

`docker compose exec web sh`

DKR-10

Images, Tags & Registries

`ghcr.io/acme/api` `:v2.3.1`

⋮ ⋮

registry / namespace / repo tag (mutable pointer)

- **Tags are mutable.** `myapp:latest` can point to a different image tomorrow — pin explicit versions in production, never deploy off `latest`.
- **Digests are immutable.** `myapp@sha256:abc123...` always resolves to the exact same bytes — use it when you need a guarantee.
- **docker.io is the default registry** if none is specified; private registries (ECR, GCR, GHCR, Harbor) need `docker login` first.

```
REGISTRY

docker build -t ghcr.io/acme/api:v2.3.1 .
docker push ghcr.io/acme/api:v2.3.1
docker pull ghcr.io/acme/api:v2.3.1
docker tag myapp:latest ghcr.io/acme/api:v2.3.1
```

DKR-11

Isolation: Namespaces & cgroups

Two separate Linux kernel features do the actual isolating — Docker just wires them together per container.

MECHANISM	CONTROLS
PID namespace	Container sees only its own process tree (its main process is PID 1 inside).
NET namespace	Own network stack — interfaces, routing table, ports.
MNT namespace	Own filesystem mount view — the image's layered filesystem.
UTS namespace	Own hostname, independent of the host's.
USER namespace	Maps container UIDs to different (often unprivileged) host UIDs.
cgroups	Caps CPU, memory, and I/O a container can consume — enforced by the kernel, not just requested.

LIMITS

```
docker run --memory=512m --cpus=1.5 myapp
```

REMINDER

Namespaces provide isolation, not a security boundary as strong as a VM's. Don't run untrusted code in containers with the same trust assumptions as a hypervisor.

DKR-12

Dockerfile Best Practices

- ▶ **Use small base images.** `-slim` or `-alpine` variants cut image size and attack surface versus full distro images.
- ▶ **Order instructions by change frequency.** Dependencies before source code, so code edits don't invalidate the dependency-install cache layer.
- ▶ **One process per container.** Don't run a supervisor managing five services inside one container — compose multiple containers instead.
- ▶ **Don't run as root.** Add a `USER` instruction; a compromised process shouldn't have root inside the container.
- ▶ **Use a `.dockerignore`.** Keep `node_modules`, `.git`, and build artifacts out of the build context — smaller context, faster builds.
- ▶ **Pin versions.** `FROM node:20.11-slim`, not `FROM node` — reproducible builds beat surprises.
- ▶ **Multi-stage for compiled languages.** Ship the binary, not the toolchain that built it.
- ▶ **Combine RUN steps thoughtfully.** Chaining related commands with `&&` avoids leaving stray layers with cache/apt artifacts.

IMAGES & CONTAINERS

```
# images
docker build -t app:1.0 .
docker images
docker rmi app:1.0
docker image prune -a

# containers
docker run -d -p 8080:80 --name web app:1.0
docker ps -a
docker stop web && docker rm web
docker exec -it web sh
```

INSPECT & CLEAN

```
# inspect / debug
docker logs -f web
docker inspect web
docker stats
docker top web

# cleanup
docker system prune -a --volumes
docker volume ls
docker network ls
docker container prune
```

docker run --rm -it app sh

docker cp web:/app/log.txt .

docker diff web

docker events